

PowerShell – Automatisation et Administration Windows

R4.05

Historique de PowerShell

- **2006 – Windows PowerShell 1.0**
Créé par Microsoft pour remplacer les scripts batch et VBScript, basé sur **.NET** et le **modèle objet**.
- **2016 – Windows PowerShell 5.1**
Version intégrée par défaut dans Windows, largement utilisée pour l'administration système.
- **2018 – PowerShell Core 6**
Réécriture multiplateforme (Windows, Linux, macOS) basée sur **.NET Core**.
- **Aujourd'hui – PowerShell 7+**
Version unifiée, moderne, performante et multiplateforme, utilisée aussi bien en administration que dans le cloud (Azure, DevOps).

PowerShell est aujourd'hui l'outil standard d'automatisation de l'administration Windows.

Ressources officielles

<https://learn.microsoft.com/fr-fr/powershell/>



Les commentaires

Les commentaires servent à :

- Expliquer le code
- Faciliter la maintenance
- Améliorer la lisibilité
- Documenter les scripts

Commentaire sur une ligne

```
# Ceci est un commentaire
```

Commentaire sur plusieurs lignes

```
<#  
Ceci est un  
commentaire  
multiligne  
#>
```

Structure d'une commande PowerShell ou cmdlet

- Principe général :

Une commande PowerShell (cmdlet) suit la forme : **Verbe-Nom**

- Exemple:

Get-Process

Variables

- Le nom d'une variable commence par le symbole « \$ » (ex: \$maVariable)
- N'est pas sensible à la casse
- Peut contenir des caractères spéciaux (non recommandé)
- Peut même contenir des espaces (non recommandé)

Dans ce cas, le nom de la variable doit être placé entre accolades (« {} »)

- (ex: `${ma variable}`)

Déclarer une variable

Pour déclarer une variable, il suffit de la nommer, suivie du symbole " = ", puis de sa valeur

Exemples :

```
$var1= 12
```

Afficher une variable

Pour afficher une variable, il suffit d'écrire son nom

On peut aussi utiliser la commande " Write-Output »"(echo)

```
$var1
```

```
Write-Output $var1
```

```
$PSVersionTable # version de powershell
```


Les types

- La valeur d'une variable possède un type
- Le type détermine les opérations qui peuvent être effectuées sur la variable

(on peut multiplier deux nombres, mais on ne peut pas multiplier deux chaînes de caractères!)

Exemples :

```
$var1= 12
```

```
$var1.GetType() # affiche son type
```

Type	Nom du type .NET	Description	Exemple
String	System.String	Chaîne de caractères (texte)	"Bonjour"
Integer	System.Int32	Nombre entier	42
Long	System.Int64	Grand nombre entier	1234567890
Double	System.Double	Nombre décimal	3.14
Boolean	System.Boolean	Vrai ou faux	\$true, \$false
DateTime	System.DateTime	Date et heure	Get-Date
Array	System.Object[]	Collection ordonnée d'éléments	1, 2, 3
Hashtable	System.Collections.Hash table	Paires clé / valeur	@{Nom="Alice"; Age=30}
Object	System.Object / PSObject	Objet PowerShell (résultat de commandes)	Get-Process
Null	\$null	Absence de valeur	\$null

Chaîne de caractères (string ou System.String)

```
$maVariable = "Bonjour le monde!"
```

```
$maVariable = 'Bonjour le monde!'
```

```
# Lorsqu'on utilise les guillemets (""), on peut inclure  
d'autres variables dans notre chaîne
```

```
$prenom = "Gilles"
```

```
$salutation = "Bonjour $prenom!"
```

```
# Bonjour Gilles!
```

```
$salutation = 'Bonjour $prenom!'
```

```
# Bonjour $prenom!
```

Booléen

Prend soit la valeur « \$true » (pour « vrai ») ou la valeur « \$false » (pour « faux »)

`$monBooléen = $true`

Conversion de types

On peut convertir une variable d'un type à l'autre

```
$maVariable = "42"
```

Cette variable est de type string, même si la chaîne de caractères ne contient que des chiffres

```
$maVariableConvertie = [int]$maVariable
```

Nous avons converti la chaîne de caractères « "42" » vers le nombre entier « 42 »

constantes

Une constante, c'est une variable qui ne change jamais de valeur

Ex: la consante π en mathématiques.Utile pour...

- Éviter de répéter une valeur qu'on utilise à plusieurs endroits dans le code
- Rendre le code plus facile à comprendre en évitant d'utiliser des valeurs non documentées (valeurs « hardcodées »)
- Il est de pratique courante d'écrire les constantes toutes en majuscules avec des "_" entre les mots

```
New-Variable -Name PI -Value 3.14 -Option Constant
```

Affectation manuelle de types et transtypage

```
[int]$var = 12
```

```
[int]$nombre = Read-Host 'Entrez un nombre'
```

Entrez un nombre : coucou

Impossible de convertir la valeur « coucou » en type « System.Int32 ».

Erreur : « Le format de la chaîne d'entrée est incorrect. »

Au caractère Ligne:2 : 1

```
+ [int]$nombre = Read-Host 'Entrez un nombre'
```

```
+ ~~~~~
```

```
+ CategoryInfo          : MetadataError: (:) [],  
ArgumentTransformationMetadataException
```

```
+ FullyQualifiedErrorId : RuntimeException
```

Portée des variables

La **portée** d'une variable détermine **où elle est accessible** dans un script ou une session PowerShell.

- **Types de portées**
- **Local** (*par défaut*) : visible dans le bloc courant (fonction, script)
- **Script** : visible dans tout le script
- **Global** : visible dans toute la session PowerShell
- **Private** : limitée strictement à la portée actuelle

Portée des variables

```
$global:var = "Global"  
function Test {  
    $local:var = "Local"  
    Write-Output $var  
}
```

Bonnes pratiques :

- Privilégier la portée **Locale**
- Limiter l'usage de **Global**
- Éviter les conflits de noms

Opérateurs

Les opérateurs permettent d'effectuer des opérations sur des variables

Types d'opérateur:

- Arithmétiques
- D'affectation
- Unaires
- De comparaison
- Logiques
- De chaînes de caractères

Opérateurs arithmétiques

$\$a + \b : Addition

$\$a - \b : Soustraction

$\$a * \b : Multiplication

$\$a / \b : Division

$\$a \% \b : Modulo

Retourne le reste d'une division, ex: $5 \% 3 = 2$

Exemple d'utilisation: vérifier si un nombre est pair ou impair

Opérateurs d'affectation

`$a = 3`

`$a += 3` # incrémente la variable de 3

`$a = 5`

`$a += 3` # la valeur de \$a est maintenant 8

`$a -= 3`

`$a *= 3`

`$a /= 3`

`$a %= $b`

Opérateurs logiques

```
$a -ge 5 -and $a -le 10
```

```
# $a est plus grand ou égal à 5 et $a est plus petit ou  
égal à 10
```

```
$a -lt 5 -or $a -gt 10
```

```
# $a est plus petit que 5 ou est plus grand que 10
```

```
# Il ne s'agit pas d'un « ou exclusif »
```

```
not ($a -lt $b)
```

```
# Opérateur de négation
```

```
# Dans ce cas, équivalent à « $a -ge $b »
```

```
# Peut être combiné avec des -and et des -or
```

Opérateurs de chaînes de caractères

`$a + $b` : concatène `$a` et `$b`

Exemple :

`$a = "Mon chat"`

`$b = " est très tannant"`

`$c = $a + $b`

`# La valeur de $c est "Mon chat est très tannant"`

Opérateur de format -f

L'opérateur -f permet de formater une chaîne de caractères en y insérant des valeurs.

Principe

Les emplacements sont indiqués par {index} Les valeurs sont fournies après -f

```
"Bonjour {0}, aujourd'hui nous sommes le {1}" -f  
"Alice", (Get-Date)
```

```
# Bonjour Alice, aujourd'hui nous sommes le 24/01/2026  
12:47:18
```

Le système d'aide PowerShell

Basé sur la commande Get-Help

Fournit :

- Description
- Syntaxe
- Paramètres
- Exemples

Fonctionne pour :

- Cmdlets
- Scripts
- Fonctions
- Providers

La commande Get-Help

Syntaxe de base :

```
Get-Help Nom-De-La-Commande
```

Exemple :

```
Get-Help Get-Process -Detailed
```

```
Get-Help Get-Process -Full
```

```
Get-Help Get-Process -Examples
```

```
Get-Help Get-Process -Online
```

```
Get-Command Get-* # Trouver des cmdlets liées à un verbe
```

```
Get-Help *service* # Trouver une commande par mot-clé
```

```
Update-Help # Mettre à jour l'aide, à faire en administrateur
```

Principaux paramètres

-Name / -Identity : Spécifie l'objet ciblé (fichier, service, processus...)

`Get-Service -Name spooler`

-Path : Indique le chemin d'accès

`Get-ChildItem -Path C:\Windows`

-Recurse : Applique l'action aux sous-dossiers

`Remove-Item C:\Test -Recurse`

-Force : Force l'exécution (fichiers cachés, protégés...)

`Remove-Item fichier.txt -Force`

-WhatIf : Simule la commande sans rien modifier

`Remove-Item fichier.txt -WhatIf`

-Confirm : Demande une confirmation avant d'exécuter

`Stop-Service spooler -Confirm`

-Verbose : Affiche des détails supplémentaires

`Copy-Item a.txt b.txt -Verbose`

PowerShell : un langage orienté objet

- Principe fondamental

Contrairement à d'autres shells, PowerShell manipule des **objets**, pas du texte.

Les commandes retournent :

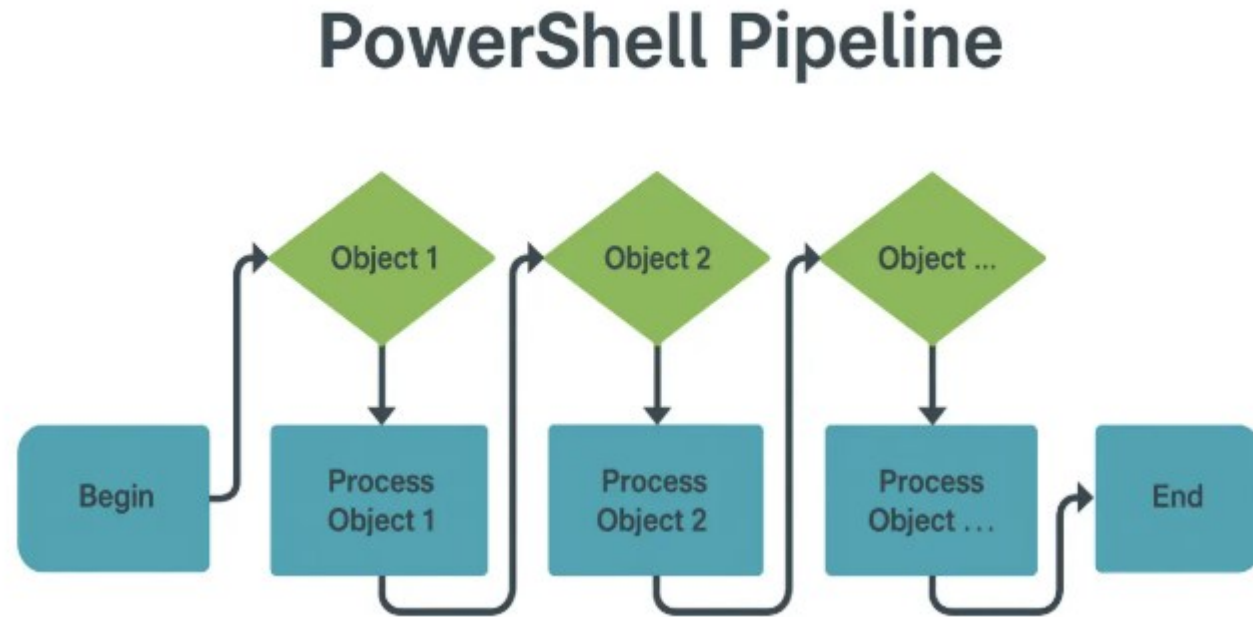
- des objets .NET
- avec des propriétés
- et des méthodes

Le pipeline

- Symbole : |
- La sortie est un objet
- Permet de chaîner les commandes

Exemple :

```
Get-Service | where-Object Status -eq 'Running'
```



Rappel : les flux

Standard input (stdin), standard output (stdout), and standard error (stderr)

Stream #	Description	Cmdlet	Common Parameters
1	Success	Write-Output	None – this is the default output
2	Error	Write-Error	-ErrorAction and -ErrorVariable
3	Warning	Write-Warning	-WarningAction and -WarningVariable
4	Verbose	Write-Verbose	-Verbose
5	Debug	Write-Debug	-Debug
6	Information	Write-Information	-InformationAction and -InformationVariable

Variable \$_ ou \$PSItem

- Représente l'objet courant dans le pipeline
- Utilisée dans Where-Object, ForEach-Object...
- Exemple :

```
1..3 | ForEach-Object {  
    "Extérieur: $PSItem"  
    1..2 | ForEach-Object {  
        "Intérieur: $PSItem"  
    }  
}
```

Liaisons

- En PowerShell, lorsqu'on appelle une **cmdlet** ou une **fonction**, les arguments passés sont **associés aux paramètres**.
- Il existe deux principales façons de faire ce "binding" (association) :

Type	Description
ByValue	PowerShell regarde le type de l'objet et l'associe directement à un paramètre du même type .
ByPropertyName	PowerShell regarde les propriétés de l'objet et les associe aux paramètres portant le même nom.

Fonctionnement de ByValue

- PowerShell regarde le type de l'objet passé en argument.
- Il cherche un paramètre dont le **type** correspond exactement ou est compatible.
- Si trouvé → la valeur est associée directement à ce paramètre.

Exemple classique : Processus → Arrêt

`Get-Process notepad` | `Stop-Process`

`Get-Process notepad` → retourne des objets

`ProcessStop-Process` → accepte ces objets par valeur (-InputObject)

Les processus Notepad sont arrêtés

Aucun besoin de préciser le nom ou l'ID : l'objet est transmis tel quel.

`Stop-Process` possède un paramètre : -InputObject <Process[]>

Comparaison rapide avec ByPropertyName

`Get-Process notepad` | `Stop-Process -Id $_.Id`

Fonctionnement de ByPropertyName

ByPropertyName est une méthode de “parameter binding” dans PowerShell.

- Ici, PowerShell ne regarde pas le type de l’objet, mais les noms des propriétés de l’objet.
- Si une propriété de l’objet a le même nom qu’un paramètre, sa valeur est automatiquement assignée.

```
Get-Random | Stop-Process
```

#Erreur

```
New-Object -TypeName PSObject -Property @{ 'Id' = (Get-Random)} | Stop-Process -WhatIf
```

Exemple simple : Services → Arrêt par nom

`Get-Service` | `Stop-Service`

Ce qui se passe :

1. `Get-Service` renvoie des objets avec la propriété `Name`
2. `Stop-Service` possède le paramètre `-Name`
3. PowerShell fait la correspondance `Name` → `-Name`
4. Les services sont arrêtés ➡ Ici, ce n'est pas du `ByValue`, mais du `ByPropertyName`

Attention :

`Get-Random` | `Stop-Process`

`#Erreur`

`New-Object -TypeName PSObject -Property @{ 'Id' = (Get-Random) } | Stop-Process -WhatIf`

Qu'est-ce qu'un objet ?

Un **objet** est composé de :

- **Propriétés** → informations (nom, statut, taille...)
- **Méthodes** → actions possibles

Get-Service

Chaque service est un **objet**.

```
PS /home/nickp> Get-Process
```

						Property
NPM(K)	PM(M)	WS(M)	CPU(s)	Id	SI	ProcessName
0	0.00	3.52	0.10	47	46	bash
0	0.00	0.32	0.09	1	1	init
0	0.00	0.22	0.00	46	46	init
0	0.00	172.23	25.38	4514	46	pwsh
0	0.00	115.63	18.65	4678	46	pwsh
0	0.00	2.65	0.04	4677	46	sudo

Collection

Object

Propriétés d'un objet

- Afficher les propriétés :

```
Get-Service | Select-Object Name, Status
```

- Accéder à une propriété :

```
$service = Get-Service -Name Spooler  
$service.Status
```

Méthodes d'un objet

- Lister méthodes et propriétés :

`$service` | `Get-Member`

- Exemple de méthode :

`$texte = "powershell"`

`$texte.ToUpper()`

`# ToUpper() est une méthode`

Création d'un objet à partir de rien

Méthode principale : [PSCustomObject]

```
$utilisateur = [PSCustomObject]@{
```

```
    Nom          = "Dupont"
```

```
    Prenom       = "Alice"
```

```
    Age          = 30
```

```
    Service      = "Informatique"
```

```
}
```

```
$utilisateur | Add-Member -NotePropertyMembers  
@{DateNaissance=[DateTime] '08/26/2004' }
```

```
$utilisateur
```

```
Nom          : Dupont  
Prenom       : Alice  
Age          : 30  
Service      : Informatique  
DateNaissance : 26/08/2004 00:00:00
```

Les collections

Certaines cmdlet fournissent plusieurs types d'objets :

```
$result = Get-Childitem C:\Windows
```

```
$result | Get-Member
```

```
# TypeName : System.IO.DirectoryInfo
```

```
# TypeName : System.IO.FileInfo
```


Collection d'objets

Get-Member, lorsqu'appliqué à une **variable contenant une collection d'objets**, retourne les **membres des objets contenus dans la collection** (un par type).

Get-Member s'applique donc dans ce contexte sur le contenu.

Pour que Get-Member s'applique sur le contenant, il faut l'utiliser de la façon suivante:

```
$result = Get-Childitem C:\windows  
Get-Member -InputObject $result  
# TypeName : System.Object[]
```

Premier membre d'une collection

```
$result[0]
```

```
$result[0].LastWriteTime
```

```
$result[0] | Get-Member
```

Expansion automatique des propriétés des éléments d'une collection

```
(Get-Process).Name
```

```
# Forme classique
```

```
Get-Process | where { $_.id -gt 1000 }
```

```
# est équivalent à :
```

```
# Forme simplifiée
```

```
Get-Process | where id -gt 1000
```

Sélection/récupération des résultats

Paramètre	Description
-Property <Object[]>	Spécifie les propriétés à sélectionner
-ExcludeProperty <string[]>	Supprime les propriétés spécifiées de la sélection
-ExpandProperty <string>	Spécifie une propriété à sélectionner et indique qu'une tentative doit être faite pour développer cette propriété
-First <int>	Spécifie le nombre d'objets à sélectionner au début d'un tableau d'entrée
-Index <Int32[]>	Sélectionne les objets d'un tableau en fonction de la valeur d'index. Entrez les index dans une liste séparée par des virgules
-Last <int>	Spécifie le nombre d'objets à sélectionner à la fin d'un tableau d'entrée
-Skip <int>	Ignore le nombre spécifié d'éléments (commence à compter à partir de 1)
-Unique <Switch>	Spécifie que si, un sous-ensemble des objets d'entrée contient des objets ayant tous des propriétés et des valeurs identiques, un seul membre de ce sous-ensemble sera sélectionné

Exemples

```
Get-Process | Select-Object -first 5
```

```
# Récupération des n premiers objets
```

```
Get-Process | Select-Object -Unique | Select-Object -First 5
```

```
# Récupération d'objets uniques
```

Récupération d'une propriété particulière : select-object

```
Get-Process | Select-Object Name, Id | Get-Member  
1,2,3,4,5,6,7,8 | Select-Object -First 2 -skip 1  
# selectionne une propriété unique d'un objet
```

Examen de tous les objets d'une collection

```
Get-Service | ForEach-Object {$_.DisplayName.ToUpper()}  
| Select-Object -First 5  
(Get-Service | ForEach-Object DisplayName).ToUpper() |  
Select-Object -first 5
```

Tri des objets : Sort-Object

```
Get-Process | Sort-Object -Property id -Descending
```

```
Get-Process | Sort-Object CPU -Descending | select-Object -First 5
```


Filtrer les objets : Where-Object

Get-Service | where-Object {\$_.Status -eq 'running'}

Get-Process | where-Object -Property CPU -gt -value 1

comparaisons

Comparison	Operator	Case-Sensitive Operator
Equality	-eq	-ceq
Inequality	-ne	-cne
Greater than	-gt	-cgt
Less than	-lt	-clt
Greater than or equal to	-ge	-cge
Less than or equal to	-le	-cle
Wildcard equality	-like	-clike

```
Get-Process | Where-Object ProcessName -eq pwsh
Get-Process | Where-Object ProcessName -like pwsh
Get-Process | Where-Object ProcessName -like *pwsh
Get-Process | Where-Object ProcessName -like *wsh
Get-Process | Where-Object ProcessName -contains pwsh
Get-Process | Where-Object ProcessName -in "pwsh", "bash"
Get-Process | Where-Object -FilterScript {$PSItem.ProcessName -eq 'pwsh' -
and $PSItem.CPU -gt 1}
```

Enumerations : Foreach-Object

```
Get-ChildItem "c:\Windows" | Foreach-Object -MemberName tostring
```

```
1..10 | Foreach-Object {Get-Random}
```

Parallel enumeration

- Execute 5 par 5 (-ThrottleLimit)
- Disponible uniquement en PowerShell 7+

```
1..10 | ForEach-Object -Parallel {  
    Start-Sleep 1  
    "Nombre $_ traité"  
} -ThrottleLimit 3
```

Comptage/mesure des objets

```
(Get-Process).length
```

```
# 335
```

```
Get-Process | Measure-Object
```

```
# Count      : 335
```

```
# Average    :
```

```
# Sum        :
```

```
# Maximum    :
```

```
# Minimum    :
```

```
# Property   :
```

Comparaison d'objets

```
$fic1= Get-Content .\fic1.txt  
$fic2 = Get-Content .\fic2.txt  
Compare-Object - ReferenceObject $fic1 -DifferenceObject  
$fic2
```

Format-Table

```
Get-Childitem C:\ | Format-Table
```

```
Get-Childitem C:\ -Force | Format-Table -Property  
mode,name,length,isreadonly,creationTime
```

Transformation d'un objet existant

```
Get-Ciminstance win32_OperatingSystem | select-Object  
@{name='ComputerName'; expression={$_.CSName}},@{n='OS';  
e={$_.Caption}},version
```

ComputerName	OS	version
-----	--	-----
Microsoft Windows	11 Professionnel	10.0.26200

Tableaux : Arrays

- Définition

```
$MyArray = @(1, 2, 3, 4)
```

```
$MyArray.GetType()
```

```
$NewArray = 1..10
```

- Accès

```
$NewArray[0]
```

```
$NewArray[0, 2, 5, 7]
```

```
$NewArray[-2]
```

Manipulation tableaux

Copie simple (superficielle) : Les deux variables pointent vers le **même tableau**

```
$tab1 = 1, 2, 3
```

```
$tab2 = $tab1
```

Copie avec Clone()

```
$tab1 = 1, 2, 3
```

```
$tab2 = $tab1.Clone()
```

Copie avec @() : méthode recommandée

```
$tab2 = @($tab1)
```

Hashtables

- Une hashtable associe :
- une **clé** (unique)
- à une **valeur**

Création d'une hashtable :

```
$hash = @{ }  
$hash = @{  
    "clé1" = "valeur1"  
    "clé2" = 42  
    "clé3" = $true  
}
```

Hashtables

- Ajouter ou modifier

```
$hash["clé4"] = "nouvelle valeur"
```

- Supprimer

```
$hash.Remove("clé2")
```

- Parcourir une hashtable

```
foreach ($key in $hash.Keys) {  
    "$key = $($hash[$key])"  
}
```

- Ou

```
$hash.GetEnumerator()
```

Cas d'usage très courant : splatting

```
$params = @{  
    Name  = "Test"  
    Force = $true  
}
```

```
New-Item @params
```

Chaque clé devient un paramètre

Les structures conditionnelles If / else

```
if (expression) {  
    # code à exécuter si l'expression est vraie  
} else {  
    # code à exécuter si l'expression est fausse  
}
```

Les structures conditionnelles If / elseif / else

```
if (expression1) {  
    # code à exécuter si expression1 est vraie  
} elseif (expression2) {  
    # code à exécuter si expression1 est fausse  
    # et expression2 est vraie  
} else {  
    # code à exécuter si les deux expressions sont  
    # fausses  
}
```

switch

```
$Array = 1,2,3,4,5  
switch ($Array) {  
    1 {write-Output '$Array contains 1'}  
    3 {write-Output '$Array contains 3'}  
    6 {write-Output '$Array contains 6'}  
}
```


Boucle For

```
for ($i=0; $i -lt 10; $i++) {  
    $i # actions  
}
```

Boucles : foreach

- Syntaxe

```
foreach ( $element in <expression> ) {<scriptblock>}
```

- Exemple

```
$names = @("Alice", "Bob", "Charlie")
```

```
foreach ($name in $names) {  
    $name  
}
```

do while and do until loop

- Syntaxe

```
do {<scriptblock>} until/while (<condition> is true)
```

- Exemple

```
$number = 0  
do { $number ++  
    write-Output "The number is $number"  
} while ($number -ne 5)
```

while

- Syntaxe

```
while (<condition> is true) {<scriptblock>}
```

- Exemple :

```
$number = 0  
while ( $number -ne 5) { $number ++  
    write-Host "The number is $number"  
}
```

loop

- Syntaxe

```
for (<Iterator> ; <condition> ; <iteration>) {  
    <scriptblock> }
```

- Exemple

```
for ($i = 0 ; $i -lt 5 ; $i ++ ) {  
    Write-Host $i  
}
```

break

```
$number = 0
while ( $number -ne 5) {
    $number ++
    if ($number -eq 3) {
        break
    }
    Write-Host "The number is $number"
}
```

Continue

```
$number = 0
while ( $number -ne 5) {
    $number ++
    if ($number -eq 3) {
        continue
    }
    Write-Host "The number is $number"
}
```

Format-List

Format-List sert à afficher les objets sous forme de liste, en montrant leurs propriétés.

```
Get-Process CalculatorApp | Format-List -Property Name, Id,  
CPU, Responding
```

```
Name      : CalculatorApp  
Id         : 80216  
CPU        : 0,375  
Responding : True
```


Format-Table

Format-Table permet d'afficher les objets sous forme de tableau, en colonnes.

```
Get-Service | Format-Table
```

```
Get-Service | Format-Table Name, Status, StartType
```

Name, Status, StartType sont des propriétés de l'objet Service.

Ajustement automatique

```
Get-Service | Format-Table -AutoSize
```

```
Get-Service | Format-Table -AutoSize
```

Forcer le retour à la ligne :

```
Get-Process | Format-Table Name, Path -Wrap
```

CSV

Get-Process | ConvertTo-Csv

Get-Process w* | ConvertTo-Csv | Out-File
C:\temp\ProcessesConvertTo.csv

Ou mieux

Get-Process | Export-Csv C:\temp\ProcessesExport.csv -
useculture

Les fichiers CSV en PowerShell

CSV = **C**omma **S**eparated **V**alues

- Fichier texte
- Données organisées en colonnes
- Séparateur : , ou ;
- Très utilisé pour l'échange de données (Excel, scripts, outils)

Exemple de fichier CSV

Nom;Age;Ville

Alice;25;Paris

Bob;30;Lyon

Importer un fichier CSV

```
$utilisateurs = Import-Csv utilisateurs.csv
```

Chaque ligne devient un **objet PowerShell**

Accès aux propriétés :

```
$utilisateurs[0].Nom
```

Filtrer :

```
$utilisateurs | where-Object Age -gt 25
```

Exporter vers un fichier CSV

```
Get-Service | Export-Csv services.csv -  
NoTypeInfo
```

-NoTypeInfo évite une ligne inutile

Spécifier le séparateur

```
Import-Csv fichier.csv -Delimiter ";"
```

```
Import-Csv fichier.csv -UseCulture
```

Les alias

Cmdlet	explication
Get-Alias	Visualise les alias déjà définis
New-Alias	Crée de nouveaux alias
Export-Alias	Exporte les alias dans un fichier
Import-Alias	Importe les alias depuis un fichier

Fonctions

Une **fonction** est un bloc de code réutilisable qui permet :

- d'organiser un script
- d'éviter les répétitions
- de rendre le code plus lisible
- Déclaration d'une fonction

```
function Nom-Fonction {  
    # Instructions  
}  
  
function Dire-Bonjour {  
    Write-Host "Bonjour PowerShell"  
}
```

- Appeler une fonction

Dire-Bonjour

Fonctions avec paramètres

```
function Calcul-AnneeNaissance {  
    param(  
        [string]$Nom,  
        [int]$Age  
    )  
  
    $annee = (Get-Date).Year - $Age  
    return "Bonjour $Nom, tu es né(e) en $annee."  
}  
$resultat = Calcul-AnneeNaissance -Nom "Alice" -Age 25  
Write-Host $resultat  
#Bonjour Alice, tu es né(e) en 2001.
```


CmdletBinding

CmdletBinding est un attribut spécial qu'on met au début d'une fonction PowerShell pour transformer cette fonction en cmdlet avancée.

```
function MaFonction {  
    [CmdletBinding()]  
    param (  
        [string]$Nom,  
        [int]$Age  
    )  
  
    write-host "Nom : $Nom, Age : $Age"  
}
```

Explications :

[CmdletBinding()] transforme la fonction en cmdlet avancée.

param() définit les paramètres comme dans une cmdlet classique.

On peut maintenant appeler la fonction avec -Nom "Pierre" -Age 21.

Avantages de CmdletBinding

- Support du pipeline

"Pierre", "Paul" | MaFonction

Support des paramètres communs-Verbose, -Debug, -ErrorAction, -WarningAction

MaFonction -Nom "Pierre" -Verbose

Validation de paramètres

```
param(  
    [Parameter(Mandatory=$true)]  
    [ValidateNotNullOrEmpty()]  
    [string] $Nom
```

)

Exemple complet

```
function Affiche-Info {  
    [CmdletBinding(SupportsShouldProcess=$true)]  
    param(  
        [Parameter(Mandatory=$true)]  
        [string] $Nom,  
  
        [int] $Age  
    )  
  
    if ($PSCmdlet.ShouldProcess($Nom, "Affichage des informations")) {  
        Write-Host "Nom : $Nom, Age : $Age"  
    }  
}
```

SupportsShouldProcess=\$true permet à la fonction de supporter -WhatIf et -Confirm : pratique pour tester des scripts qui modifient le système.
\$PSCmdlet.ShouldProcess() vérifie si l'action doit être exécutée.

BEGIN, PROCESS et END

Quand on écrit une **fonction avancée**, PowerShell nous permet de séparer le traitement en **3 étapes logiques** :

Bloc	Quand il s'exécute	Usage principal
BEGIN	Une seule fois, au début de la fonction	Initialisation, création de variables, affichage de messages d'intro
PROCESS	Une fois pour chaque élément du pipeline	Traitement principal, logique répétitive pour chaque objet reçu
END	Une seule fois, à la fin	Nettoyage, résumé, actions finales, messages de fin

Syntaxe générale

```
function MaFonction {  
    [CmdletBinding()]  
    param(  
        [Parameter(ValueFromPipeline=$true)]  
        [string]$Nom  
    )  
    BEGIN {  
        Write-Host "Début de la fonction"  
    }  
    PROCESS {  
        Write-Host "Traitement de $Nom"  
    }  
    END {  
        Write-Host "Fin de la fonction"  
    }  
}
```

"Pierre","Paul","Julie" | MaFonction
Début de la fonction
Traitement de Pierre
Traitement de Paul
Traitement de Julie
Fin de la fonction

Get-Command

CommandType	Name	Version	Source
-----	----	-----	-----
Alias	Add-AppPackage	2.0.1.0	Appx
Alias	Add-AppPackageVolume	2.0.1.0	Appx
Alias	Add-AppProvisionedPackage	3.0	Dism
Alias	Add-MsixPackage	2.0.1.0	Appx
Alias	Add-MsixPackageVolume	2.0.1.0	Appx
Alias	Add-MsixVolume	2.0.1.0	Appx
Alias	Add-ProvisionedAppPackage	3.0	Dism
Alias	Add-ProvisionedAppSharedPackageContainer	3.0	Dism
Alias	Add-ProvisionedAppxPackage	3.0	Dism
Alias	Add-ProvisioningPackage	3.0	Provisioning
Alias	Add-TrustedProvisioningCertificate	3.0	Provisioning
Alias	Apply-WindowsUnattend	3.0	Dism
Alias	Disable-PhysicalDiskIndication	2.0.0.0	Storage
Alias	Disable-PhysicalDiskIndication	1.0.0.0	VMDirectStorage
Alias	Disable-StorageDiagnosticLog	2.0.0.0	Storage
Alias	Disable-StorageDiagnosticLog	1.0.0.0	VMDirectStorage
Alias	Dismount-AppPackageVolume	2.0.1.0	Appx
Alias	Dismount-MsixPackageVolume	2.0.1.0	Appx
Alias	Dismount-MsixVolume	2.0.1.0	Appx
Alias	Enable-PhysicalDiskIndication	2.0.0.0	Storage
Alias	Enable-PhysicalDiskIndication	1.0.0.0	VMDirectStorage
Alias	Enable-StorageDiagnosticLog	2.0.0.0	Storage
Alias	Enable-StorageDiagnosticLog	1.0.0.0	VMDirectStorage
Alias	Flush-Volume	2.0.0.0	Storage
Alias	Flush-Volume	1.0.0.0	VMDirectStorage
Alias	Get-AppPackage	2.0.1.0	Appx
Alias	Get-AppPackageAutoUpdateSettings	2.0.1.0	Appx
Alias	Get-AppPackageDefaultVolume	2.0.1.0	Appx
Alias	Get-AppPackageLastError	2.0.1.0	Appx
Alias	Get-AppPackageLog	2.0.1.0	Appx

Cmdlets courantes d'administration

Get-Service
Start-Service
Get-Process
Stop-Process
Get-EventLog
Get-WinEvent
Get-ADUser
Get-ADComputer
Get-NetIPAddress

Nombre de commandes

`Get-Command -CommandType cmdlet,function | Measure-Object`

Count : 2630
Average :
Sum :
Maximum :
Minimum :
Property :

Nombre de commande avec le nom object

Get-Command -Noun object

-----	----		-----	-----
Cmdlet	Compare-Object	3.1.0.0	Microsoft.PowerShell.Utility	
Cmdlet	ForEach-Object	3.0.0.0	Microsoft.PowerShell.Core	
Cmdlet	Group-Object	3.1.0.0	Microsoft.PowerShell.Utility	
Cmdlet	Measure-Object	3.1.0.0	Microsoft.PowerShell.Utility	
Cmdlet	New-Object	3.1.0.0	Microsoft.PowerShell.Utility	
Cmdlet	Select-Object	3.1.0.0	Microsoft.PowerShell.Utility	
Cmdlet	Sort-Object	3.1.0.0	Microsoft.PowerShell.Utility	
Cmdlet	Tee-Object	3.1.0.0	Microsoft.PowerShell.Utility	
Cmdlet	where-Object	3.0.0.0	Microsoft.PowerShell.Core	

Autres commandes

Get-Command -Commandtype *alias*

Get-Command *Get-**

Get-Command **-computer*

Aide sur les commandes

Get-Help maCommande

Get-Help Get-Item -Full

Get-Help Get-Item -Examples

Get-Help Get-Item -Online

Get-Member

```
$s="Bonjour RT"
```

```
$s | Get-Member
```

```
$s.ToUpper()
```

```
$s.Length
```

Get-Alias

DOS/CMD	Équivalent PowerShell (alias)	Commandelette PowerShell	Description
dir	dir, gci, ls	Get-ChildItem	Liste le contenu d'un répertoire.
cd	cd, chdir, sl	Set-Location	Change de répertoire courant.
md	md, ni	New-Item	Crée un fichier/répertoire.
rd	del, erase, rd, ri, rm, rmdir	Remove-Item	Supprime un fichier/répertoire.
move	mi, move, mv	Move-Item	Déplace un fichier/répertoire.
ren	ren, rni	Rename-Item	Renomme un fichier/répertoire.
copy	copy, cp, cpi	Copy-Item	Copie un fichier/répertoire.

Création d'un fichier

```
New-Item -Name monFichier.txt -ItemType file -value 'vive  
PowerShell !'
```

Fournisseurs PowerShell

Toutes les commandes que nous avons vues précédemment (familles *-Item et *-Location) permettent la manipulation non seulement du système de fichiers mais aussi:

- de la base de registre,
- des variables,
- des variables d'environnement,
- des alias,
- de la base de certificats X.509 de votre ordinateur,
- des fonctions,
- et du paramétrage WSMAN (utile à la fonctionnalité Remote PowerShell).

Get-PSProvider

Get-Childitem Alias:

Get-Childitem Env:

Get-Childitem C:

Get-Childitem Function:

Get-Childitem HKCU:

Get-Childitem Variable:

Get-Childitem Cert:

Get-Childitem WSMAN:

Scripts PowerShell

- Fichiers .ps1
- Exécution locale ou distante
- ExecutionPolicy
- Bonnes pratiques : commentaires

Gestion fichiers et dossiers

- Get-ChildItem
- New-Item
- Copy-Item
- Move-Item
- Remove-Item

Accès aux ressources Windows

- Services
- Processus
- Registre
- Journaux d'événements
- Réseau

Pourquoi utiliser des scripts ?

Points clés :

- Automatisation de tâches répétitives
- Gestion centralisée de plusieurs machines
- Réduction des erreurs humaines
- Gain de temps pour les administrateurs réseau

Exemple concret :

- Sauvegarde automatique d'un dossier chaque soir
- Création de comptes utilisateurs dans Active Directory

Exécuter un script PowerShell

Fichier .ps1

Par défaut, PowerShell ne nous laisse pas exécuter de scripts!

Politiques d'exécution de PowerShell

- Restricted: PowerShell n'exécute aucun script.
- AllSigned: PowerShell exécute seulement les scripts signés
- RemoteSigned: PowerShell va refuser d'exécuter un script provenant d'Internet s'il n'est pas signé, mais il va exécuter les autres scripts sans problème.
- Unrestricted: PowerShell va accepter d'exécuter n'importe quel script, mais va demander une confirmation avant d'exécuter un script provenant d'Internet.

Pour lire la stratégie d'exécution en cours :

```
Get-ExecutionPolicy -List
```

Pour définir la stratégie pour l'utilisateur courant :

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy  
RemoteSigned
```

Pour définir la stratégie pour tous les utilisateurs de la machine :

```
Set-ExecutionPolicy -Scope LocalMachine -ExecutionPolicy  
RemoteSigned
```

Constitution d'un script

Un script est généralement constitué des parties suivantes :

Nom du script, auteur, version, etc.

Déclaration des paramètres

...

Déclaration des fonctions

...

Corps principal du script

...

Gestion des erreurs

Objectifs

- Comprendre ce qu'est une **erreur** dans un script
- Éviter l'arrêt brutal d'un script
- Rendre les scripts **plus fiables et maintenables**

Exemple de problème

1. Fichier inexistant
 2. Accès refusé
 3. Service arrêté
 4. Variable vide ou mal typée
- Sans gestion d'erreur : **script cassé**
 - Avec gestion d'erreur : **script contrôlé**

Les types d'erreurs en PowerShell

Erreurs non bloquantes (Non-Terminating)

- Le script continue
- Exemple : fichier absent avec Get-Item
- Message affiché en rouge, mais pas d'arrêt

Erreurs bloquantes (Terminating)

- Le script s'arrête immédiatement
- Exemple : division par zéro, throw

try/catch ne fonctionne que sur les erreurs bloquantes

Structure du bloc try / catch / finally

```
try {  
    # Code à surveiller  
}  
catch {  
    # Code exécuté en cas d'erreur  
}  
finally {  
    # Code exécuté dans tous les cas  
}
```

Rôles

- **try** : instructions à risque
- **catch** : réaction à l'erreur
- **finally** : nettoyage (facultatif)

Exemple simple sans gestion d'erreur

```
Get-Content "C:\data\fichier.txt"  
Write-Host "Lecture terminée"
```

Si le fichier n'existe pas :

- Message d'erreur
- Script continue → comportement imprévisible

Exemple avec try / catch

```
try {  
    Get-Content "C:\data\fichier.txt"  
    Write-Host "Lecture terminée"  
}  
catch {  
    Write-Host "Erreur : le fichier est introuvable" -ForegroundColor Red  
}
```

Avantages :

- Message clair pour l'utilisateur
- Script maîtrisé
- Pas de crash non expliqué

Forcer une erreur avec -ErrorAction Stop

```
try {  
    Get-Item "C:\data\fichier.txt" -ErrorAction Stop  
}  
catch {  
    Write-Host "Fichier non trouvé"  
}
```

-ErrorAction Stop transforme une erreur non bloquante en erreur bloquante.

Exploiter l'erreur avec \$_

```
try {  
    10 / 0  
}  
catch {  
    Write-Host "Message : $($_.Exception.Message)"  
}
```

Informations disponibles

- Message
- Type d'exception
- Ligne concernée

Utile pour le debug et les logs

Le bloc finally

```
try {  
    Write-Host "Traitement..."  
}  
catch {  
    Write-Host "Erreur détectée"  
}  
finally {  
    Write-Host "Fin du script"  
}
```

finally est toujours exécuté , Idéal pour :

- Fermer un fichier
- Libérer une ressource
- Écrire dans un journal

Bonnes pratiques

- ✓ Toujours gérer les actions critiques
- ✓ Afficher des messages clairs
- ✓ Utiliser -ErrorAction Stop si nécessaire
- ✗ Ne pas masquer les erreurs sans message
- ✗ Ne pas utiliser try/catch partout inutilement

Accès distant avec PowerShell

PowerShell Remoting

- Enable-PSRemoting
- Invoke-Command
- Administration centralisée

Gestion du parefeu

PowerShell fournit le module NetSecurity pour gérer le pare-feu Windows.

Get-NetFirewallRule → lister les règles

New-NetFirewallRule → créer une règle

Set-NetFirewallRule → modifier une règle

Remove-NetFirewallRule → supprimer une règle

Gestion du parefeu

Autoriser le port 80 (HTTP) :

```
New-NetFirewallRule -DisplayName "Autoriser HTTP" `
                    -Direction Inbound `
                    -Protocol TCP `
                    -LocalPort 80 `
                    -Action Allow
```

Modifier une règle existante

```
Set-NetFirewallRule -DisplayName "Autoriser HTTP" -Enabled False
```

Supprimer une règle

```
Remove-NetFirewallRule -DisplayName "Autoriser HTTP"
```

Utilisation du réseau avec PowerShell

PowerShell permet de :

- Tester la connectivité réseau
- Obtenir des informations IP
- Gérer les connexions réseau
- Diagnostiquer des problèmes réseau

Utilisation du réseau avec PowerShell

```
Test-Connection google.com
```

```
Test-Connection google.com -Count 2 # limiter le nombre de requêtes
```

```
Get-NetAdapter # Afficher les interfaces réseau
```

```
Get-NetIPAddress # Afficher les adresses IP
```

```
Get-NetIPConfiguration # configuration complète
```

```
Get-NetRoute
```

```
Test-NetConnection google.com -Port 443 # Tester un port
```

```
Resolve-DnsName google.com # Résolution DNS
```

```
Get-NetTCPConnection | Where-Object State -eq "Established"
```

```
# Filtrer par état
```

```
Invoke-WebRequest -Uri "https://example.com" -OutFile "fichier.html"
```

```
# Téléchargement de fichiers
```

netevent

Windows fournit un ensemble de cmdlets PowerShell basées sur ETW (Event Tracing for Windows)

Les principales sont

`New-NetEventSession`

`Add-NetEventPacketCaptureProvider`

`Add-NetEventNetworkAdapter`

`Start-NetEventSession`

`Stop-NetEventSession`
`Remove-NetEventSession`

Elles permettent de capturer du trafic réseau, un peu comme pktmon, mais de façon plus scriptable et PowerShell-friendly.

Exemple simple de capture réseau en PowerShell

Dans une console administrateur

Créer une session

```
New-NetEventSession -Name CaptureReseau -LocalFilePath C:\Temp\capture.etl
```

Ajouter un provider de capture de paquets

```
Add-NetEventPacketCaptureProvider -SessionName CaptureReseau
```

Démarrer la capture

```
Start-NetEventSession -Name CaptureReseau
```

... laisser tourner ...

Arrêter la capture

```
Stop-NetEventSession -Name CaptureReseau
```

Conclusion

- Automatisation fiable
- Gain de temps
- Sécurité et traçabilité
- Compétence clé R&T